

KECERDASAN BUATAN

Polynomial Regression



Nama : Hesekiel Josafat Marbun

NIM : 42324022

D3 Teknologi Informasi

**INSTITUT TEKNOLOGI DEL
FAKULTAS VOKASI
2026/2027**

A. Penjelasan Bagian Latihan (Praktikum)

Cell In [1] : Memuat Library dan Dataset

```
#In [1]
import pandas as pd
path = "Ice_cream_selling_data.csv"
df = pd.read_csv(path)
print(df.head())
```

[2] ✓ 0.0s Python

	Temperature (°C)	Ice Cream Sales (units)
0	-4.662263	41.842986
1	-4.316559	34.661120
2	-4.213985	39.383001
3	-3.949661	37.539845
4	-3.578554	32.284531

Penjelasan : mengimpor library pandas untuk manipulasi tabel (dataframe). Model memuat file CSV berisi data penjualan es krim dan menampilkan 5 baris pertamanya (head()). Data ini berisi 2 kolom: Temperature (Suhu) dan Ice Cream Sales (Penjualan).

Cell In [2] : Memisahkan Fitur (X) dan Target (y)

```
#In [2]
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

[3] ✓ 0.0s Python

Penjelasan : Fungsi .iloc digunakan untuk memotong data berdasarkan indeks baris dan kolom.

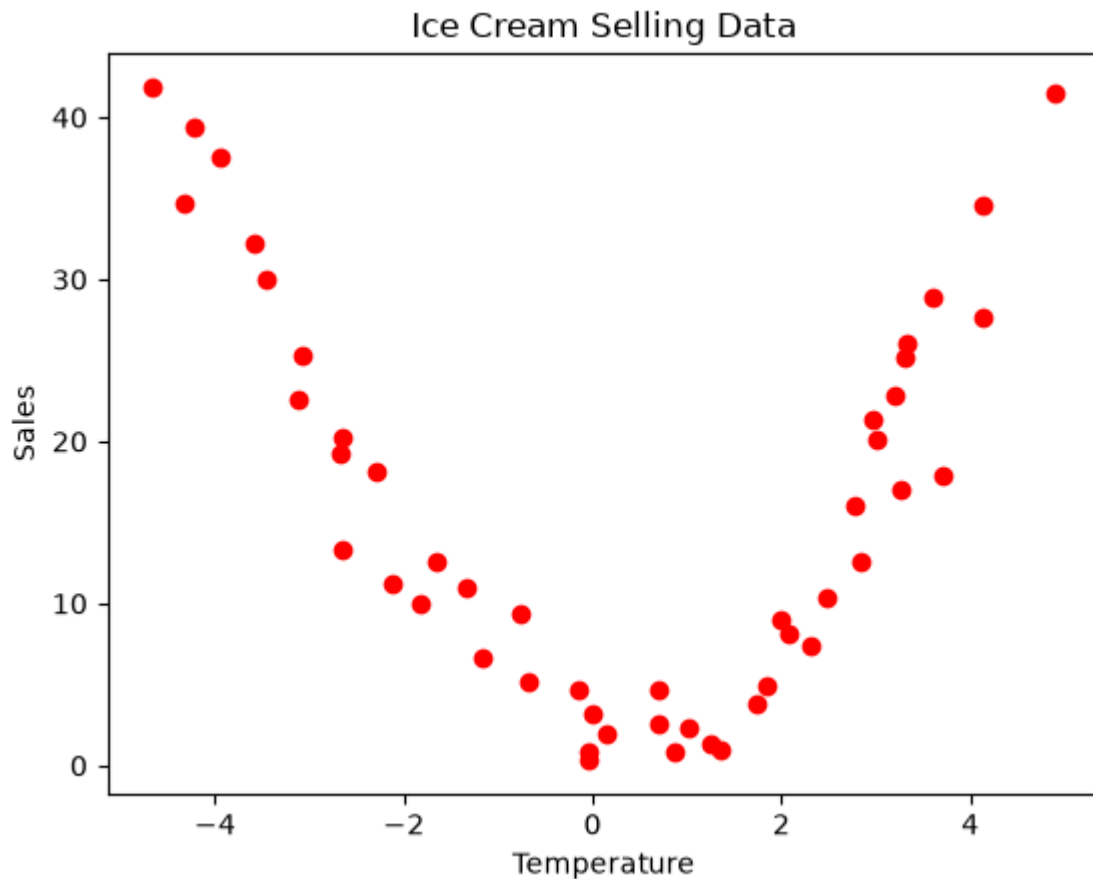
- `[:, :-1]` artinya mengambil semua baris (:), dan semua kolom *kecuali kolom paling terakhir* (:-1) sebagai Variabel X (Prediktor / Suhu).
- `[:, -1]` artinya mengambil semua baris pada kolom *terakhir saja* sebagai Variabel y (Target / Angka Penjualan).

Cell In [3] : Visualisasi Data Asli

```
#In [3]
import matplotlib.pyplot as plt
plt.scatter(X, y, color = 'red')
plt.title('Ice Cream Selling Data')
plt.xlabel('Temperature')
plt.ylabel('Sales')
plt.show
```

[5] ✓ 1.3s Python

... <function matplotlib.pyplot.show(close=None, block=None)>



Penjelasan : menggunakan matplotlib untuk menggambar *scatter plot* (titik-titik). Sumbu X adalah suhu, sumbu y adalah penjualan. Dari grafik yang dihasilkan, terlihat bentuknya melengkung menyerupai kurva berbentuk 'U' atau parabola, **bukan garis lurus**.

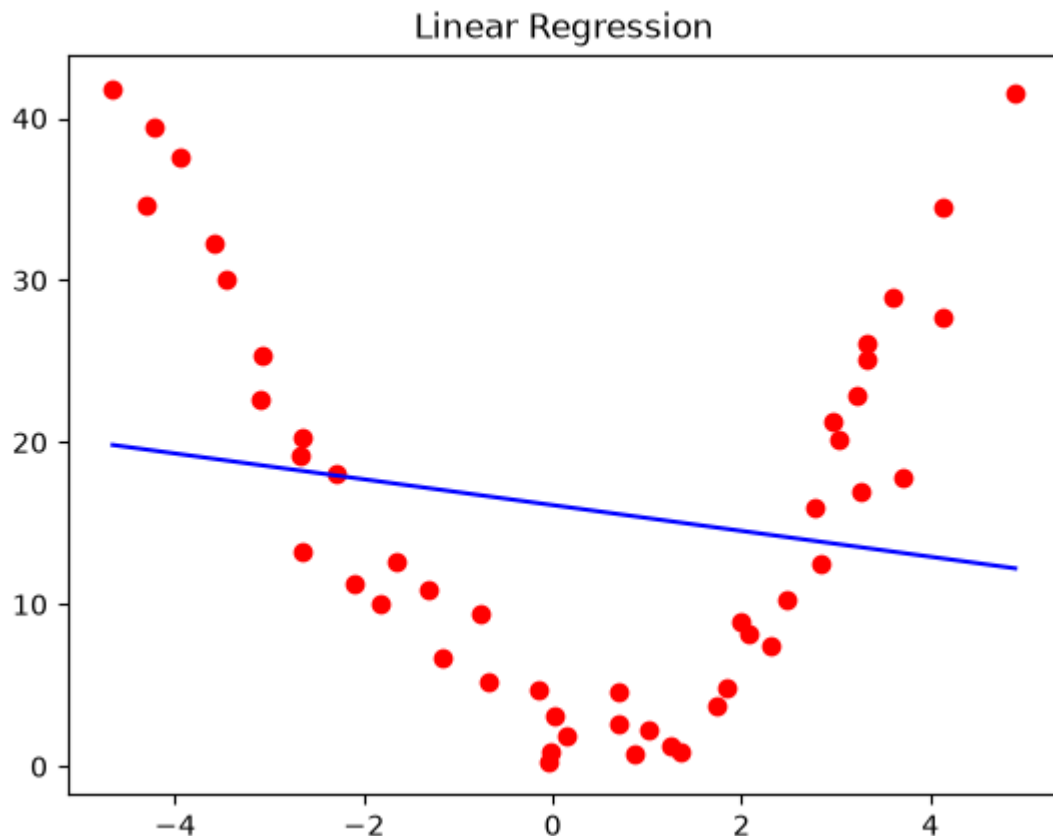
Cell In [4] : Mencoba Linear Regression Biasa (Derajat 1)

```
#In [4]
from sklearn.linear_model import LinearRegression
linear_model = LinearRegression()
linear_model.fit(X,y)
plt.scatter(X, y, color = 'red')
plt.plot(X, linear_model.predict(X), color = 'blue')
plt.title('Linear Regression')
```

[7] ✓ 2.9s

... Text(0.5, 1.0, 'Linear Regression')

Python



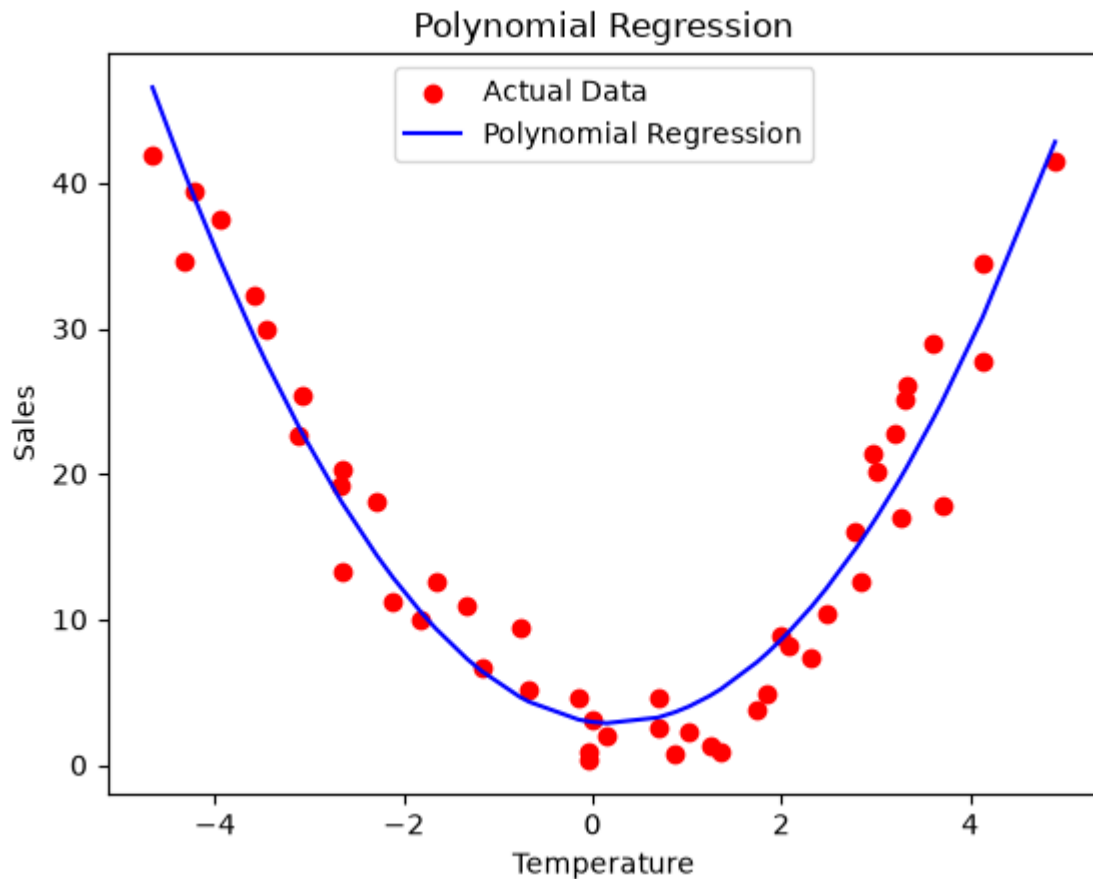
Penjelasan : Karena pakai Linear Regression biasa (garis lurus), model dilatih menggunakan `.fit(X, y)`. Ketika hasilnya diplot (garis biru), garis lurus ini gagal mengikuti pola titik merah yang melengkung. Kondisi ini disebut **Underfitting**.

Cell In [5] : Menggunakan Polynomial Regression (Satu Prediktor)

```
#In [5]
from sklearn.preprocessing import PolynomialFeatures
# Buat Polynomial Features
poly_reg = PolynomialFeatures(degree=2)
X_poly = poly_reg.fit_transform(X)
# Buat model Linear Regression
linear_model2 = LinearRegression()
linear_model2.fit(X_poly, y) # Latih model dengan data polynomial
# Gunakan predict() untuk mendapatkan y_pred_poly
y_pred_poly = linear_model2.predict(X_poly)
# Plot Polynomial Regression
plt.scatter(X, y, color='red', label="Actual Data")
plt.plot(X, y_pred_poly, color='blue', label="Polynomial Regression")
plt.title('Polynomial Regression')
plt.xlabel('Temperature')
plt.ylabel('Sales')
plt.legend()
plt.show()
```

[9] ✓ 0.2s

Python



Penjelasan : Karena garis lurus gagal, maka perlu menaikkan dimensi fungsi menjadi kuadratik (X^2) dengan mensetting `degree=2`.

- `fit_transform(X)` akan mengubah variabel X (misal [X]) menjadi array polinomial [1, X, X^2].
- Setelah X berubah menjadi polinomial (X_{poly}), kita masukkan ke algoritma `LinearRegression` biasa.

Cell In [6] : Menampilkan Persamaan Matematis Polinomial

```
#In [6]
print('Intercept: ', linear_model2.intercept_)
print('Coefficient: ', linear_model2.coef_)
print('Equation: y = ', linear_model2.intercept_, '+', linear_model2.coef_[1], 'x +', linear_model2.coef_[2], 'x^2')

[10] ✓ 0.0s Python

... Intercept: 2.9517741579934427
Coefficient: [ 0.         -0.82468167  1.82952623]
Equation: y = 2.9517741579934427 + -0.824681670210649 x + 1.8295262296702088 x^2
```

Penjelasan : Mencetak nilai bias (*Intercept*) dan koefisien dari setiap derajat x. Persamaan akhirnya terbentuk menjadi: $Y = 2.95 + (-0.82)X + 1.82X^2$.

Cell In [7] & In [8] : Beralih ke Data Multivariabel (Banyak Prediktor)

```
#In [7]
dataset = pd.read_csv('manufacturing.csv')
print(dataset.head())
```

```
[12] ✓ 0.0s Python
```

	Temperature (°C)	Pressure (kPa)	Temperature x Pressure \
0	209.762701	8.050855	1688.769167
1	243.037873	15.812068	3842.931469
2	220.552675	7.843130	1729.823314
3	208.976637	23.786089	4970.736918
4	184.730960	15.797812	2918.345014

	Material Fusion Metric	Material Transformation Metric	Quality Rating
0	44522.217074	9.229576e+06	99.999971
1	63020.764997	1.435537e+07	99.985703
2	49125.950249	1.072839e+07	99.999758
3	57128.881547	9.125702e+06	99.999975
4	38068.201283	6.303792e+06	100.000000

```
#In [8]
X = dataset.iloc[:, 2:5].values
y = dataset.iloc[:, 5].values
X
```

```
[13] ✓ 0.0s Python
```

```
array([[1.68876917e+03, 4.45222171e+04, 9.22957596e+06],
       [3.84293147e+03, 6.30207650e+04, 1.43553672e+07],
       [1.72982331e+03, 4.91259502e+04, 1.07283887e+07],
       ...,
       [3.95630467e+03, 6.26576910e+04, 1.40595745e+07],
       [4.97723476e+03, 5.71959855e+04, 9.13403620e+06],
       [3.97489712e+03, 4.10923929e+04, 4.35891253e+06]], shape=(3957, 3))
```

Penjelasan : Membaca dataset kedua (manufacturing.csv). Indeks 2:5 pada variabel X mengambil 3 kolom secara bersamaan (indeks 2, 3, dan 4) untuk memprediksi indeks 5 (Quality Rating).

Cell In [9] : Membagi Data Training dan Testing

```
#In [9]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[14] ✓ 0.0s Python
```

Penjelasan : Data dipecah: 80% data (X_train, y_train) digunakan untuk melatih (*training*) model, dan 20% data (X_test, y_test) disembunyikan untuk menguji (*testing*) seberapa pintar modelnya nanti.

Cell In [10] : Melatih Model Multivariabel Polinomial

```
#In [10]
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
poly_reg = PolynomialFeatures(degree=2)
X_train_poly = poly_reg.fit_transform(X_train)
X_test_poly = poly_reg.transform(X_test)
linear_model2 = LinearRegression()
linear_model2.fit(X_train_poly, y_train)
y_pred = linear_model2.predict(X_test_poly)
```

```
[15] ✓ 0.0s Python
```

Penjelasan : Model dilatih, dan setelahnya disuruh menebak/memprediksi (predict) menggunakan data test (X_test_poly). Hasil tebakan disimpan di variabel y_pred.

Cell In [11] : Evaluasi Akurasi (R-Square Score)

```
#In [11]
from sklearn.metrics import r2_score
score=r2_score(y_test,y_pred)
score
```

```
[17] ✓ 0.0s Python
```

```
0.9095053064574692
```

Penjelasan : Fungsi `r2_score` membandingkan kunci jawaban asli (`y_test`) dengan tebakan model (`y_pred`). Nilai maksimal adalah 1.0 (100% akurat).

B. Penjelasan Bagian TUGAS

```
#Tugas
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# 1. Membaca dataset
dataset = pd.read_csv('manufacturing.csv')

# 2. Menentukan variabel X (predictor) dan y (target) sesuai halaman 5
X = dataset.iloc[:, 2:5].values
y = dataset.iloc[:, 5].values

# 3. Membagi data menjadi train dan test set
# Tambahkan random_state agar hasil split tetap konsisten
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Melakukan perulangan untuk derajat 1, 3, dan 5
derajat_list = [1, 3, 5]

print("--- Hasil R-Square Score ---")
for derajat in derajat_list:
    # Buat Polynomial Features
    poly_reg = PolynomialFeatures(degree=derajat)
    X_train_poly = poly_reg.fit_transform(X_train)
    X_test_poly = poly_reg.transform(X_test) # Gunakan transform, BUKAN fit_transform untuk data test

    # Buat dan latih model
    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    # Prediksi
    y_pred = model.predict(X_test_poly)

    # Hitung R-square
    score = r2_score(y_test, y_pred)
    print(f"Polynomial Derajat {derajat} : {score:.5f}")
```

[16] ✓ 0.0s Python

```
...
--- Hasil R-Square Score ---
Polynomial Derajat 1 : 0.46249
Polynomial Derajat 3 : 0.80499
Polynomial Derajat 5 : 0.90951
```

1. Definisikan list derajat yang diminta asisten praktikum: `derajat_list = [1, 3, 5]`.
2. Gunakan *looping* (perulangan) `for derajat in derajat_list`.
3. Di dalam loop, dijalankan:
 - `poly_reg = PolynomialFeatures(degree=derajat)` -> Angka 2 diganti dengan variabel derajat (1, lalu 3, lalu 5).
 - Ubah data X train dan X test jadi polinomial.
 - Latih model (`fit`).
 - Tebak hasil uji (`predict`).
4. Lalu jalankan kode :
 - Hitung `r2_score` dan langsung dicetak ke layar (`print()`).

Maka didapat :

- 1. Polynomial Derajat 1 (Score: 0.46249) – *Underfitting***
- 2. Polynomial Derajat 3 (Score: 0.80499) - *Good Fit (Optimal)***
- 3. Polynomial Derajat 5 (Score: 0.90951) - *High Complexity & Potensi Overfitting***